Microsoft Purview Managed Service

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview Managed Service 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Scope boundary . . . 5

Outcome . . . 5

1. Solution Overview — Microsoft Purview Managed Service 5

What this delivers . . . 5

Why a managed service . . . 5

How it works . . . 5

What you can control . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Microsoft Purview Managed Service 7

2.1 Microsoft 365 + Purview licensing for the capabilities being operated . . . 7

2.2 Least-privilege operational roles with privacy role separation . . . 7

2.3 A defined service scope and SLO/cadence targets . . . 7

2.4 The owning capability solutions identified (this service coordinates, not re-implements) . . . 7

2.5 Monitoring & alert configuration available . . . 8

2.6 Change control and a rollback discipline for tuning . . . 8

2.7 Stakeholder reporting and escalation path . . . 8

Checklist . . . 8

Global Requirements — Microsoft Purview Solutions 9

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 9

2. Purview roles (least-privilege) and portal access . . . 9

3. Security & Compliance PowerShell + Graph toolchain . . . 9

4. Data sources & scope in scope . . . 10

5. Compliance, records, privacy & legal stakeholders . . . 10

6. A pilot scope and change control . . . 10

3. Implementation Guide — Microsoft Purview Managed Service 11

Step 1 — Baseline the estate (Assess) . . . 12

Step 2 — Design the operating model (Design) . . . 12

Step 3 — Configure monitoring and triage (Configure) . . . 12

Step 4 — Pilot the cadence (Pilot) . . . 13

Step 5 — Validate (Validate) . . . 13
Step 6 — Run continuously and improve (Expand) . . . 13
5. Testing Guide — Microsoft Purview Managed Service 14
Gate 1 — Monitoring baseline captured . . . 14
Gate 2 — Capability-to-owner map complete . . . 14
Gate 3 — Cadence calendar and runbooks defined . . . 14
Gate 4 — Monitoring sweep returns actionable signals . . . 15
Gate 5 — Triage assigns severity and an owner . . . 15
Gate 6 — Tuning is piloted safely in the owning capability . . . 15
Gate 7 — Sensitive surfaces reported as posture only . . . 15
Gate 8 — SLOs met on the pilot . . . 15
Gate 9 — Service report reconciles and is owned . . . 15
Gate 10 — Improvement loop closes . . . 15
6. Troubleshooting Guide — Microsoft Purview Managed Service 16
The monitoring baseline looks empty for a surface . . . 16
The monitoring sweep returns no activity . . . 16
A cmdlet is unavailable (e.g. insider risk) . . . 17
Triage is too noisy or too slow (SLOs missed) . . . 17
A finding has no owner / isn't routed . . . 17
Someone tried to enforce a high-impact change from the managed service . . . 17
A tuning change disrupted work . . . 17
Insider-risk / communication-compliance detail is showing up . . . 18
The service report diverges from the portal . . . 18
SLO adherence can't be computed . . . 18
A determination item was actioned before governance signed off . . . 18
7. Best Practices — Microsoft Purview Managed Service 19
8. Deliverables Summary — Microsoft Purview Managed Service 20
Documentation . . . 20
PowerShell scripts . . . 20
Named artifacts (verbatim — used across docs, scripts, and tests) . . . 21
What the buyer receives . . . 21
Outcome . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary — Microsoft Purview Managed Service

White-label · Purview Solutions

Microsoft Purview is not "set and forget." Once labels, DLP, retention, insider risk, and audit are deployed, the estate needs **continuous operation** — someone to watch for drift, triage what the alerts surface, tune policies safely, report to stakeholders, and keep improving. Left unstaffed, policies drift into stale or stuck states, coverage decays as new workloads appear, alerts pile up unreviewed, and the next audit finds a program that quietly degraded. Most teams can't staff that continuous operation — the work is real but intermittent.

This is a complete **managed-service operating model** for Microsoft Purview: the runbooks, cadences, monitoring, triage, tuning, reporting, and improvement loop needed to **run Purview continuously**. It sits **on top of** the individual capability solutions and **operates** them — it does not re-implement them. It is written so an administrator can stand up a repeatable, low-overhead operating model with clear SLOs and a safe-change discipline.

What you get

• A plain-language Solution Overview of the operating model and how monitoring, triage, tuning, reporting,

and the improvement loop work together.

• A Requirements Guide with verify / implement-if-missing for every managed-service prerequisite —

including the service scope, SLO/cadence targets, and the capability-to-owner map.

• An Implementation Guide that stands up the baseline, runbooks, cadences, monitoring/triage, a piloted

cadence, and continuous operation.

• Five PowerShell scripts for the monitoring baseline, a monitoring/triage sweep, a health/validation

check, safe remediation, and the service-report rollup.

• Testing and Troubleshooting guides for a confident rollout.

What this delivers

A named, reusable operating model — a `Managed Service – Monitoring Baseline`, a `Managed Service – Capability Map`, `Managed Service – SLO Targets`, a `Managed Service – Cadence Calendar`, a `Managed Service – Run Book`, a `Managed Service – Triage Queue`, a `Managed Service – Tuning Log`, a `Managed Service – Service Report`, and a `Managed Service – Improvement Backlog` — run on a defined cadence with high-impact changes piloted in their owning capability and a documented rollback.

The standard deployment model

Assess → Design → Configure → Pilot (the cadence) → Validate → Expand (run continuously).

Scope boundary

The service **operates and coordinates** the capability solutions; the actual high-impact changes (DLP enforce, label/retention changes, insider risk) are delivered and changed by **their** solutions, piloted in simulation/test. **Regulatory, records, retention-schedule, legal-hold, and insider-risk determinations remain with compliance/records/privacy/legal.** HR/privacy-sensitive surfaces are reported posture only.

Outcome

Microsoft Purview run continuously — drift caught early, activity triaged with severity and an owner, tuning piloted safely, a regular reconciled service report, and an improvement backlog that trends the estate healthier — proven on a piloted cadence before it scales to the full service.

1. Solution Overview — Microsoft Purview Managed Service

What this delivers

This solution provides a **managed-service operating model for Microsoft Purview** — the repeatable runbooks, cadences, monitoring, triage, tuning, reporting, and improvement loop needed to **run Purview continuously** rather than deploy it once and let it drift. It is the "keep it healthy and improving" layer that sits **on top of** the individual capability solutions (DLP, sensitivity labels, retention/ records, insider risk, audit, data governance, the health check). It does **not** re-implement those capabilities — it **operates** them: watching the estate, triaging what happens, tuning safely, reporting to stakeholders, and feeding an improvement backlog.

Why a managed service

- **Purview is not "set and forget."** Policies drift, coverage decays, alerts pile up, labels go stale,

and new workloads appear — without an operating cadence, the estate quietly degrades between projects.

- **Most teams can't staff continuous operation.** The work is real but intermittent; a defined operating

model with runbooks and cadences makes it repeatable and low-overhead.

- **Safe change beats heroics.** High-impact changes (DLP enforce, label/retention changes, insider risk)

are piloted in **simulation/test** in their owning capability — the managed service coordinates and evidences, it does not flip switches recklessly.

- **Reporting turns activity into assurance.** A regular service report shows stakeholders the estate is

monitored, incidents are triaged, and the program is improving.

How it works

1. **Establish a monitoring baseline** — inventory the estate and its operational signals (policy modes,

alert configuration, audit status, coverage) as the reference the service watches for drift.

1. **Run the cadences** — daily monitoring, regular **triage** of DLP/alert activity into a queue,

periodic **tuning** (safe, piloted), and a scheduled **service review**.

1. **Operate with runbooks** — named runbooks for alert triage, DLP tuning, health monitoring, and onboarding a new workload, each with clear steps and a change/rollback discipline.

1. **Report and improve** — a periodic **service report** (health, activity, SLO adherence) plus an **improvement backlog** that turns findings into planned, owned work.

What you can control

- The cadence definitions (what runs daily/weekly/monthly/quarterly) and the SLO targets.
- The monitoring baseline and what counts as drift.
- The triage severity model and the routing of each finding to its owning capability.
- The service-report contents and the improvement backlog.

Benefits

- **Continuous health** — drift, stuck policies, and coverage decay are caught early, not at audit time.
- **Fast, consistent triage** — DLP/alert activity flows into a queue with severity and an owner.
- **Safe tuning** — changes are piloted in simulation/test in the owning capability, with rollback.
- **Stakeholder assurance** — a regular report proves the estate is operated and improving.

Limitations (be honest with buyers)

- This is an **operating model**, not a re-implementation — the underlying capabilities (DLP, labels, retention, insider risk, audit) are delivered and changed by **their** solutions, piloted in simulation/test; the managed service **coordinates, monitors, and evidences** them.

- Advanced monitoring/alerting, insider risk, and premium reporting require **Microsoft 365 E5 / E5 Compliance **— the Requirements Guide names the licensing;** verify your licensing/tenant.**

- **Regulatory, records, retention-schedule, legal-hold, insider-risk, and communication-compliance determinations remain with compliance/records/privacy/legal** — the service surfaces technical state and routes determinations; it does not make them.

- Insider-risk and communication-compliance surfaces are reported as **posture only** (HR/privacy-sensitive) — never case contents or a person's identity.

The standard deployment model

> ***Assess** → **Design** → **Configure** → **Pilot (the cadence)** → **Validate** → **Expand (run continuously)**, with high-impact changes piloted in simulation/test in their owning capability and a documented rollback.*

2. Requirements Guide — Microsoft Purview Managed Service

Verify **every** requirement before standing up the operating model. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (licensing, least-privilege roles, S&C PowerShell/Graph toolchain, in-scope data, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) — this guide adds the **managed-service-specific** prerequisites and does not repeat the six.

2.1 Microsoft 365 + Purview licensing for the capabilities being operated

Description: Licensing that covers the capabilities the service will run — labels + baseline DLP in **E3 / Business Premium**; **advanced DLP/alerts, auto-labeling, insider risk, Audit Premium, premium Compliance Manager reporting** in **E5 / E5 Compliance**. **Why:** The service operates whatever the tenant is licensed for; monitoring/alerting depth is plan- gated. **Verify:** M365 admin center → **Billing** → **Licenses**; run `OPSMANAGED-Discovery.ps1` to see what the tenant exposes. **Verify your licensing/tenant** — availability evolves. **Implement:** Confirm the plan for each operated capability; where a capability is unlicensed, record it as out of scope (a licensing decision), not a service gap.

2.2 Least-privilege operational roles with privacy role separation

Description: The **least-privilege roles** the service operators need to monitor, triage, and tune — e.g. read roles for monitoring, **Compliance Administrator / DLP / Information Protection / Records Management** for tuning within change control — with HR/privacy-sensitive roles (Insider Risk, Communication Compliance) held by **separate** owners and used posture-only for monitoring. **Why:** Purview uses fine-grained role groups; a managed service expects least privilege and separation of duties, especially on HR/privacy-sensitive surfaces. **Verify:** `https://purview.microsoft.com` → **Settings** → **Roles and scopes** → **Role groups**; confirm membership and separation. **Implement:** Assign least-privilege operational roles under change control; keep privacy-sensitive roles separate; the service reads those surfaces as **posture only**.

2.3 A defined service scope and SLO/cadence targets

Description: An agreed **service scope** (which Purview capabilities and workloads the service operates) and the **SLO / cadence targets** — what runs daily/weekly/monthly/quarterly, and the response targets for triage (e.g. High-severity DLP alert triaged within N business hours). **Why:** A managed service is only meaningful against a defined scope and measurable targets; without them, "operated" can't be evidenced. **Verify:** A one-page service definition lists the in-scope capabilities, the cadences, and the SLO targets, signed off by the service owner. **Implement:** Agree the scope, cadences, and SLOs with stakeholders before running the model.

2.4 The owning capability solutions identified (this service coordinates, not re-implements)

Description: Identification of the **capability solutions** that own each configuration the service operates — DLP, sensitivity labels, retention/records, insider risk, audit, data governance, and the health check — so

tuning/remediation is **routed** to the owning solution (and piloted in simulation/test there), not performed ad hoc here. **Why:** The managed service **operates and coordinates**; the actual high-impact changes belong to the owning capability, piloted first. Avoiding duplicate/conflicting changes is essential. **Verify:** Each in-scope capability maps to its owning solution and a named owner. **Implement:** Record the capability-to-owner map before running the tuning cadence.

2.5 Monitoring & alert configuration available

Description: The **alert / activity signals** the service watches — DLP alerts, alert policies, **Activity explorer**, and the **unified audit log** — configured and readable so monitoring and triage have data. **Why:** Monitoring and triage read alert/activity/audit data; without it there is nothing to operate. **Verify:** Purview → **Data Loss Prevention** → **Alerts** and **Audit** return data; alert policies exist; run `OPSMANAGED-Monitor.ps1`. **Implement:** Enable audit logging and configure the relevant alert policies (in their owning capability) before running the monitoring cadence.

2.6 Change control and a rollback discipline for tuning

Description: A documented **change-control / rollback** process for any tuning the service coordinates — switching a DLP policy from audit to enforce, adjusting a rule, turning auto-labeling out of simulation, or enabling a policy — performed in the owning capability, piloted in simulation/test first. **Why:** Operational tuning touches production, high-impact policies; every change needs an approver, a pilot, and a rollback. **Verify:** A change ticket template exists with approver, pilot scope, and rollback; the service records each change in the tuning log. **Implement:** Adopt the change process; run remediation `-WhatIf` first; make changes in the owning capability under change control.

2.7 Stakeholder reporting and escalation path

Description: A named **service owner**, a **reporting audience** (compliance/security leadership), and an **escalation path** for High-severity findings and determinations — so the service report lands with someone and incidents/decisions are routed. **Why:** A managed service that reports to nobody and can't escalate changes nothing; determinations must reach the right governance function. **Verify:** The service owner, report audience, escalation path, and the compliance/records/privacy/legal routing for determinations are named. **Implement:** Establish the reporting/escalation model before the first service review.

Checklist

- M365 + Purview licensing for each operated capability (E3/Business Premium; **E5 / E5 Compliance** for advanced) — `OPSMANAGED-Discovery.ps1`
- Least-privilege operational roles + privacy role separation (posture-only on HR/privacy surfaces)
- Service scope + SLO/cadence targets defined and signed off
- Capability-to-owning-solution map (this service coordinates, not re-implements)
- Monitoring/alert/audit signals configured and readable — `OPSMANAGED-Monitor.ps1`
- Change control + rollback discipline for tuning (piloted in owning capability)
- Service owner, reporting audience, escalation path, and determination routing named

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Microsoft Purview Managed Service

Follow these steps **in order**. This solution stands up an **operating model** — runbooks, cadences, monitoring, triage, tuning, reporting, and an improvement loop — for Microsoft Purview (`https://purview.microsoft.com`). It **operates** the capability solutions; it does not re-implement them. Portal paths and cmdlets are current at authoring; Purview evolves and features are licensing-/cloud-dependent — **verify your licensing/tenant.**

> ***Model:*** *Baseline the estate, design the operating model (runbooks + cadences + SLOs), configure monitoring/triage, **pilot the cadence** on a limited scope, validate the SLOs are met, then run continuously. High-impact changes are **piloted in simulation/test in their owning capability** — the service coordinates and evidences them. Pattern: **Assess** → **Design** → **Configure** → **Pilot (the cadence)** → **Validate** → **Expand (run continuously).***
>
> ***Determinations stay with governance.*** *Regulatory, records, retention-schedule, legal-hold, insider-risk, and communication-compliance determinations are compliance/records/privacy/legal's. The service surfaces technical state and routes determinations; it does not make them. HR/privacy-sensitive surfaces are reported **posture only**.*

The named artifacts created below are reused verbatim in Testing, Troubleshooting, and the scripts:

Artifact	Name
Monitoring baseline	`Managed Service – Monitoring Baseline`
Runbook set	`Managed Service – Run Book` (Alert Triage, DLP Tuning, Health Monitoring, New-Workload Onboarding)
Cadence calendar	`Managed Service – Cadence Calendar` (Daily / Weekly / Monthly / Quarterly)
SLO definition	`Managed Service – SLO Targets`
Triage queue	`Managed Service – Triage Queue`
Tuning log	`Managed Service – Tuning Log`
Service report	`Managed Service – Service Report`
Improvement backlog	`Managed Service – Improvement Backlog`
Capability-to-owner map	`Managed Service – Capability Map`
Pilot scope	`Managed Service - Pilot Scope`

Step 1 — Baseline the estate (Assess)

1. Run `OPSMANAGED-Discovery.ps1` (read-only) to inventory the estate and its operational signals: DLP

policies and **modes**, sensitivity labels + label policies, retention/lifecycle policies, unified- audit-log status, alert-policy presence, insider-risk **presence/posture** only, and role groups.

1. Record the result as `Managed Service – Monitoring Baseline` — the reference the service watches for

drift.

1. Build `Managed Service – Capability Map` — map each in-scope capability to its **owning solution** and

a named owner (Requirement 2.4). **Gate:** baseline captured and every capability has an owner.

Step 2 — Design the operating model (Design)

1. Define `Managed Service – SLO Targets` (Requirement 2.3) — e.g. High-severity DLP alert triaged within

N business hours; monitoring run daily; tuning reviewed monthly; service review quarterly.

1. Build `Managed Service – Cadence Calendar`:

• **Daily** — monitoring sweep (`OPSMANAGED-Monitor.ps1`); confirm audit on, no policy unhealthy.

• **Weekly** — triage the `Managed Service – Triage Queue`; route each item to its owning capability.

• **Monthly** — tuning review (safe changes, piloted in the owning capability); refresh the report.

• **Quarterly** — service review with stakeholders; re-baseline; groom the improvement backlog.

1. Write `Managed Service – Run Book` with named runbooks:

• **Alert Triage** — read alert/activity, assign severity, route to owner, record in the triage queue.

• **DLP Tuning** — propose a change, pilot it in simulation/test in the owning DLP solution, validate,

record in `Managed Service – Tuning Log`, enforce under change control.

• **Health Monitoring** — run the monitoring sweep, compare to the baseline, flag drift.

• **New-Workload Onboarding** — extend coverage to a new workload via its owning capability, then add

it to the baseline.

Step 3 — Configure monitoring and triage (Configure)

1. Confirm the alert/activity/audit signals are readable (Requirement 2.5); run `OPSMANAGED-Monitor.ps1`

to pull recent DLP/alert/activity into the `Managed Service – Triage Queue` with severity and workload.

1. Set the triage severity model (e.g. High = audit-log off, enforce-mode DLP false positive storm,

missing coverage on a regulated workload; Medium/Low below that) and the routing to each owning capability.

1. Establish `Managed Service – Service Report` and `Managed Service – Improvement Backlog` templates.

Step 4 — Pilot the cadence (Pilot)

1. Run the **daily monitoring** and **weekly triage** cadences against `Managed Service - Pilot Scope`

for one full cycle.

1. Exercise the **DLP Tuning** runbook end to end on one low-risk item — proposed, **piloted in

simulation/test in the owning DLP solution**, validated, recorded in the tuning log, then enforced under change control (or rolled back).

1. **Gate:** the cadence runs, findings flow into the queue with severity/owner, and the SLO targets are

met on the pilot. If the cadence is noisy or slow, tune the severity model/cadence before expanding.

Step 5 — Validate (Validate)

1. Complete the Testing Guide against the pilot.

2. Run `OPSMANAGED-Validation.ps1` (audit on, no intended policy missing/disabled/stuck, alert signals

readable, roles separated) and `OPSMANAGED-Report.ps1` (service report + SLO adherence).

1. **Gate:** all pass/fail gates met; the service report reconciles with the portal (authoritative);

every triage item has an owner and severity.

Step 6 — Run continuously and improve (Expand)

1. Turn on the full `Managed Service – Cadence Calendar` across the agreed service scope.

2. Each cycle: monitor → triage → route/tune (in the owning capability, piloted) → report → groom the

`Managed Service – Improvement Backlog`. Record every change in `Managed Service – Tuning Log`.

1. **Trend the health.** Keep each cycle's `Managed Service – Service Report`; a falling count of drift/

open findings and steady SLO adherence is the evidence the estate is operated and improving.

1. **Route determinations, don't make them.** Records/retention/legal/insider-risk items go to their

governance owner; the service surfaces state and evidence only.

Order of operations

`Baseline + capability map → SLOs + cadence + runbooks → monitoring/triage configured → pilot the cadence (tuning piloted in owning capability) → validate SLOs → run continuously + improvement backlog`

Rollback summary

Change	Rollback
A tuning change disrupted work	It was piloted in the owning capability — revert per that solution's rollback (DLP back to test, auto-labeling to simulation, disable a policy)
Cadence too noisy	Adjust the severity model / cadence frequency; re-pilot
A determination item was actioned early	Halt; route to compliance/records/privacy/legal
Monitoring mis-scored a licensing gap as drift	Re-tag as a licensing decision; re-run `OPSMANAGED-Report.ps1`

5. Testing Guide — Microsoft Purview Managed Service

Run these gates after the operating model is configured and piloted on `Managed Service - Pilot Scope` (Implementation Steps 3–4), and again when you expand to the full service scope. Each gate has an explicit **pass/fail** signal. The service is largely **read-and-coordinate**; the gates confirm the cadence runs, findings are triaged with severity and an owner, tuning is piloted safely in the owning capability, and the SLOs are met. The **portal is authoritative**.

Gate 1 — Monitoring baseline captured

Do: Run `OPSMANAGED-Discovery.ps1`; confirm `Managed Service – Monitoring Baseline` records DLP policies + modes, labels + policies, retention, audit status, alert-policy presence, IRM presence/posture, and role groups. **Pass:** The baseline is complete and reconciles with a portal spot-check. **Fail:** A surface is empty when the portal shows objects → a **read-access** finding; fix access and re-baseline.

Gate 2 — Capability-to-owner map complete

Do: Review `Managed Service – Capability Map`. **Pass:** Every in-scope capability maps to its **owning solution** and a named owner; nothing the service operates is unowned. **Fail:** An operated capability has no owner → assign one before running the tuning cadence.

Gate 3 — Cadence calendar and runbooks defined

Do: Review `Managed Service – Cadence Calendar` and `Managed Service – Run Book`. **Pass:** Daily/weekly/monthly/quarterly cadences are defined, and the Alert Triage, DLP Tuning, Health Monitoring, and New-Workload Onboarding runbooks exist with steps and a change/rollback discipline. **Fail:** A cadence or runbook is missing → complete it before piloting.

Gate 4 — Monitoring sweep returns actionable signals

Do: Run `OPSMANAGED-Monitor.ps1`. **Pass:** Recent DLP/alert/activity is pulled into `Managed Service – Triage Queue` with severity and workload; the audit log is confirmed on. **Fail:** No signals when the portal shows activity → confirm audit is on and the read roles are present (**verify in tenant**).

Gate 5 — Triage assigns severity and an owner

Do: Review a cycle's `Managed Service – Triage Queue`. **Pass:** Every item has a **severity** and is **routed to its owning capability** with an owner; High- severity items are escalated per the path (Requirement 2.7). **Fail:** An item lacks severity/owner, or a High item wasn't escalated → fix the triage runbook.

Gate 6 — Tuning is piloted safely in the owning capability

Do: Exercise the **DLP Tuning** runbook on one low-risk item; confirm it was **piloted in simulation/test in the owning DLP solution**, validated, recorded in `Managed Service – Tuning Log`, then enforced under change control (or rolled back). **Pass:** The change was piloted first, evidenced in the tuning log, and reversible — **no** high-impact change was flipped on directly from the managed service. **Fail:** A DLP/label/retention/insider-risk change was made without piloting in its owning capability → stop; that belongs to the owning solution, piloted first.

Gate 7 — Sensitive surfaces reported as posture only

Do: Review how insider-risk / communication-compliance surfaces appear in the baseline/report. **Pass:** The service reports **presence + privacy posture only** — no case contents, no person's identity. **Fail:** Any case detail or identity beyond authorized role scope appears → stop; the review breached the privacy boundary (Requirement 2.2).

Gate 8 — SLOs met on the pilot

Do: Run `OPSMANAGED-Report.ps1`; compare against `Managed Service – SLO Targets`. **Pass:** The pilot cycle met the SLO targets (e.g. High-severity items triaged within the target window; monitoring ran on cadence). **Fail:** An SLO was missed → adjust cadence/severity/capacity and re-pilot.

Gate 9 — Service report reconciles and is owned

Do: Produce `Managed Service – Service Report`. **Pass:** The report (health counts, activity/alert volume, SLO adherence, open backlog) reconciles with the portal and lands with the named service owner/audience. **Fail:** The rollup diverges from the portal, or has no audience → reconcile (portal is authoritative) and route it.

Gate 10 — Improvement loop closes

Do: Review `Managed Service – Improvement Backlog` across two cycles. **Pass:** Findings become planned, owned backlog items; closed items drop off; a falling drift/open-item trend is visible. Determination items remain pending their governance owner (expected). **Fail:** Findings don't turn into owned work → fix the review cadence.

Exit criteria (all must be true to call the service operational)

- Monitoring baseline captured and reconciled (Gate 1)
- Capability-to-owner map complete (Gate 2)
- Cadence calendar + runbooks defined (Gate 3)
- Monitoring sweep returns actionable signals (Gate 4)
- Triage assigns severity + owner; High items escalated (Gate 5)
- Tuning piloted safely in the owning capability (Gate 6)
- Sensitive surfaces reported posture only (Gate 7)
- SLOs met on the pilot (Gate 8)
- Service report reconciles and is owned (Gate 9)
- Improvement loop closes; determinations routed to governance (Gate 10)

6. Troubleshooting Guide — Microsoft Purview Managed Service

Common issues when running the managed-service operating model, and how to resolve them. The service is largely **read-and-coordinate**, so most problems are **read-access**, **signal-availability**, or **routing/scoring** issues — not a change that broke something. The **portal is authoritative**; artifacts (`Managed Service – *`) and scripts are named verbatim from the Implementation Guide.

The monitoring baseline looks empty for a surface

- **Symptom:** `OPSMANAGED-Discovery.ps1` shows no DLP/labels/retention/roles where the portal has them.
- **Likely cause:** The operator lacks the **read** role for that surface (Requirement 2.2) — an empty read is not an empty estate.
- **Resolve:** Record it as a **read-access** finding; assign the least-privilege read role under change control; re-baseline. Don't over-grant write access.

The monitoring sweep returns no activity

- **Symptom:** `OPSMANAGED-Monitor.ps1` finds no DLP events / activity.
- **Likely cause: Audit logging off** (Requirement 2.5), not connected to Exchange Online, or the window is genuinely quiet / audit not yet populated.
- **Resolve:** Confirm the unified audit log is on (the script reports it); `Connect-ExchangeOnline`; widen `-Days`; allow audit to populate. A persistently-off audit log is a **High-severity** triage item routed to the Audit owner.

A cmdlet is unavailable (e.g. insider risk)

- **Symptom:** `Get-InsiderRiskPolicy` or another cmdlet is missing after a good connection.
- **Likely cause:** The capability is **portal-managed** or **plan-gated** (E5 / E5 Compliance; some preview), or the module is old.
- **Resolve:** Update the module; confirm the surface in the **portal** (insider risk is largely portal-managed — report **presence + privacy posture only**) and **verify your licensing/tenant.** An unlicensed capability is out of scope, not a service gap.

Triage is too noisy or too slow (SLOs missed)

- **Symptom:** The triage queue floods, or High-severity items miss the response window.
- **Likely cause:** The severity model is mis-tuned, the cadence is too infrequent, or capacity is short.
- **Resolve:** Re-tune the severity model (what counts as High), adjust the `Managed Service – Cadence Calendar` frequency, and re-pilot. Missing an SLO is itself a finding — record it and adjust.

A finding has no owner / isn't routed

- **Symptom:** A triage item can't be routed to an owning capability.
- **Likely cause:** The `Managed Service – Capability Map` is incomplete (Requirement 2.4).
- **Resolve:** Complete the capability-to-owner map so every operated capability has an owning solution and a named owner; unowned findings rot.

Someone tried to enforce a high-impact change from the managed service

- **Symptom:** A DLP enforce / label / retention / insider-risk change was attempted directly here.
- **Likely cause:** Confusing "operate" with "re-implement."
- **Resolve: Stop.** High-impact changes belong to the **owning capability solution**, piloted in simulation/test there. Propose the change, pilot it, validate, record it in `Managed Service – Tuning Log`, then enforce under change control. `OPSMANAGED-Remediation.ps1` only enables a disabled label/retention policy and never enforces DLP.

A tuning change disrupted work

- **Symptom:** A change coordinated by the service caused blocked/disrupted work.
- **Likely cause:** It wasn't piloted in simulation/test first, or an exception was missed.
- **Resolve: Roll back per the owning solution's rollback** (DLP back to test, auto-labeling to simulation, disable a policy). Re-pilot with the exception fixed. The tuning log should show the change and its rollback.

Insider-risk / communication-compliance detail is showing up

• **Symptom:** More than presence/posture appears for these surfaces.

• **Likely cause:** The review crossed the **HR/privacy boundary** (Requirement 2.2).

• **Resolve: Stop.** Report **presence + privacy posture only** — never case contents or a person's identity. The scripts do not read case contents; keep the portal review posture-only too.

The service report diverges from the portal

• **Symptom:** `OPSMANAGED-Report.ps1` rollup doesn't match the portal.

• **Likely cause:** A read gap, a stale triage CSV, or a mis-scored row.

• **Resolve:** The **portal is authoritative** — reconcile the report to it, fix any read-access finding, ensure every open item has an owner, and re-issue to the named audience.

SLO adherence can't be computed

• **Symptom:** `OPSMANAGED-Report.ps1` reports it can't compute adherence.

• **Likely cause:** The triage CSV lacks **Opened/Closed** timestamps (or **Owner**) columns.

• **Resolve:** Add `Opened`, `Closed`, and `Owner` columns to the triage queue so the report can measure response times and ownership.

A determination item was actioned before governance signed off

• **Symptom:** A records/retention/legal-hold/regulatory/insider-risk item was changed by the service.

• **Likely cause:** A determination was treated as a technical fix.

• **Resolve: Halt**; route it to compliance/records/privacy/legal. The service surfaces technical state and evidence, not the determination.

Key reminders

Rule	Why
Operate, don't re-implement	High-impact changes belong to the owning capability, piloted first
Empty read ≠ empty estate	An unreadable surface is a read-access finding, not drift
The portal is authoritative	Reconcile the report to it before issuing
Posture only on IRM/CC	HR/privacy-sensitive — presence + guardrails, never content
Read-only + safe writes only	Monitoring/reporting read-only; only safe, logged, reversible fixes here
Determinations to governance	Records/retention/legal/regulatory/insider-risk decisions aren't the service's

7. Best Practices — Microsoft Purview Managed Service

• **Operate, don't re-implement.** The managed service **runs** the capability solutions (DLP, labels,

retention, insider risk, audit, governance, the health check) — it doesn't duplicate them. Keep a clean `Managed Service – Capability Map` so every change is routed to its owning solution and piloted there.

• **Baseline first, then watch for drift.** Capture `Managed Service – Monitoring Baseline` and score

every monitoring sweep against it. Drift (a policy stuck in test, coverage decay, audit turned off) is what the service exists to catch early.

• **Define SLOs and cadences — and hold to them.** A managed service is only credible against measurable

targets. Set `Managed Service – SLO Targets` and run the `Managed Service – Cadence Calendar` on schedule; a missed cadence is itself a finding.

• **Triage with severity and an owner.** Every item in `Managed Service – Triage Queue` gets a severity

and a routed owner; High-severity items are escalated immediately. Findings without an owner rot.

• **Tune safely — pilot in the owning capability.** Never flip a high-impact change (DLP enforce, label/

retention change, insider risk) directly from the managed service. Propose it, **pilot it in simulation/test in the owning solution**, validate, record it in `Managed Service – Tuning Log`, then enforce under change control — with a rollback.

• **Read-only by default; write under change control.** Monitoring and reporting are read-only; the only

changes are safe, piloted, reversible, and logged. Run remediation `-WhatIf` first.

• **Respect the privacy line.** Report insider-risk / communication-compliance surfaces as **posture

only** — presence and privacy guardrails, never case contents or a person's identity. Keep those roles separated.

• **Report to reconcile with the portal.** `Managed Service – Service Report` must match what the portal

shows — the portal is authoritative. A regular, reconciled report is the assurance stakeholders buy.

• **Close the improvement loop.** Turn findings into planned, owned `Managed Service – Improvement

Backlog` items; groom them each quarter. A falling drift/open-item trend is the proof the service works.

• **Route determinations to governance.** Records/retention/legal-hold/regulatory/insider-risk decisions

go to compliance/records/privacy/legal — the service surfaces technical state and evidence, not the determination.

• **Onboard new workloads deliberately.** Use the New-Workload Onboarding runbook — extend coverage via

the owning capability, then add the workload to the baseline so it's monitored from then on.

• **Verify licensing before promising a capability.** Advanced monitoring/alerting, insider risk, and premium reporting need E5 / E5 Compliance — **verify your licensing/tenant**; an unlicensed capability is out of scope, not a service gap.

8. Deliverables Summary — Microsoft Purview Managed Service

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What the managed-service operating model is — monitoring, triage, tuning, reporting, improvement — how it operates (not re-implements) the capability solutions, benefits and limitations
2. Requirements Guide	Every managed-service prerequisite with Verify / Implement-if-missing (licensing, operational roles, service scope + SLOs/cadences, capability-to-owner map, monitoring signals, change control, reporting/escalation)
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Ordered steps: baseline + capability map, design the operating model (SLOs/cadences/runbooks), configure monitoring/triage, pilot the cadence (tuning piloted in owning capability), validate SLOs, run continuously + improve
5. Testing Guide	Ten pass/fail gates from baseline through triage, safe tuning, posture-only, SLOs, reporting, and the improvement loop
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations for running Purview continuously

PowerShell scripts

Script	What it does
`OPSMANAGED-Discovery.ps1`	Estate + operational-signal inventory (the monitoring baseline): DLP policies + modes, labels + policies, retention, audit status, alert-policy presence, IRM presence/posture, role groups; read-only

Script	What it does
`OPSMANAGED-Monitor.ps1`	Monitoring/triage sweep: pulls recent DLP/alert/activity from the unified audit log into a triage view grouped by workload/severity, confirms audit is on; read-only
`OPSMANAGED-Validation.ps1`	Health/validation checker: pass/fail lines — audit on, no intended policy missing/disabled/stuck, alert signals readable, roles present — for the pilot and each cycle; exit code
`OPSMANAGED-Remediation.ps1`	Safe remediation (default `-WhatIf`): enables a disabled label/retention policy; **never** auto-enforces DLP or any high-impact change — those are routed to the owning capability and piloted there
`OPSMANAGED-Report.ps1`	Service-report rollup: policy-health counts, alert/activity volume, and SLO-adherence input from a cycle's triage CSV; exports the service report / baseline for the next cycle

Named artifacts (verbatim — used across docs, scripts, and tests)

- `Managed Service – Monitoring Baseline`, `Managed Service – Capability Map`, **`Managed Service

– SLO Targets**, Managed Service – Cadence Calendar` (Daily/Weekly/Monthly/Quarterly)

- `Managed Service – Run Book` (Alert Triage · DLP Tuning · Health Monitoring · New-Workload

Onboarding), `Managed Service – Triage Queue`, `Managed Service – Tuning Log`

- `Managed Service – Service Report`, `Managed Service – Improvement Backlog`

What the buyer receives

- This operating-model guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable operating model following the model: **Assess → Design → Configure → Pilot (the cadence)

→ Validate → Expand (run continuously)**, with high-impact changes piloted in their owning capability and a documented rollback.

Outcome

Microsoft Purview **run continuously** — a monitoring baseline watched for drift, DLP/alert activity triaged with severity and an owner, tuning piloted safely in the owning capability, a regular service report reconciled with the portal, and an improvement backlog that trends the estate healthier over time. **High-impact changes are piloted in their owning capability solution; regulatory, records, legal, and insider-risk determinations remain with compliance/records/privacy/legal.**

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OPSMANAGED-Discovery.ps1

Purpose: Connects to Security & Compliance PowerShell (and Exchange Online for the unified-audit-log status) and inventories the estate the managed service operates: DLP policies with their Mode, sensitivity labels + label policies, retention/lifecycle policies + retention labels, unified-audit-log status, insider-risk policy PRESENCE + privacy posture (posture only), and compliance role groups. This is `Managed Service – Monitoring Baseline` — the reference the service watches for drift. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Discovery.ps1 -CsvPath .\opsmanaged-baseline.csv Prereqs: ExchangeOnlineManagement (Connect-IPPSSession + Connect-ExchangeOnline), MFA-capable read admin. Verify capability availability against your licensing/tenant. EXPECTED RESULTS A read-only baseline: DLP (Name/Mode/Enabled), label + label-policy counts, retention + labels, unified-audit-log ON/OFF, IRM presence/posture note, role-group membership. Save it as the baseline the daily monitoring sweep compares against. .NOTES White-label. Read-only. Security & Compliance PowerShell / Exchange Online. No credentials in code. > [CmdletBinding()] param([string] $CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Discovery.ps1 -CsvPath .\opsmanaged-baseline.csv Prereqs: ExchangeOnlineManagement (Connect-IPPSSession + Connect-ExchangeOnline), MFA-capable read admin. Verify capability availability against your licensing/tenant. EXPECTED RESULTS A read-only baseline: DLP (Name/Mode/Enabled), label + label-policy counts, retention + labels, unified-audit-log ON/OFF, IRM presence/posture note, role-group membership. Save it as the baseline the daily monitoring sweep compares against. .NOTES White-label. Read-only. Security & Compliance PowerShell / Exchange Online. No credentials in code. > [CmdletBinding()] param([string] $CsvPath)

OPSMANAGED-Monitor.ps1

Purpose: The daily monitoring sweep for the managed service. Confirms the unified audit log is on, then pulls recent DLP events (and a general activity sample) from the unified audit log, groups them by workload, and emits a triage view for `Managed Service – Triage Queue`. It also flags any DLP policy stuck in test mode (drift signal). Read-only — it surfaces signals to triage; it changes nothing. Insider-risk / communication-compliance surfaces are NOT read here (posture only, handled in the portal). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Monitor.ps1 -Days 1 powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Monitor.ps1 -Days 7 -CsvPath .\opsmanaged-triage.csv Prereqs: ExchangeOnlineManagement; MFA-capable read admin; audit logging enabled (Requirement 2.5). EXPECTED RESULTS Audit-log ON confirmation, a per-workload DLP-event count, an activity sample, and any test-mode DLP drift flags — the raw material for the triage queue. Optionally exported to CSV. .NOTES White-label. Read-only. Security & Compliance PowerShell / Exchange Online. No credentials in code. > [CmdletBinding()] param([int] $Days = 1, [string] $CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Monitor.ps1 -Days 1 powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Monitor.ps1 -Days 7 -CsvPath .\opsmanaged-triage.csv Prereqs: ExchangeOnlineManagement; MFA-capable read admin; audit logging enabled (Requirement 2.5). EXPECTED RESULTS Audit-log ON confirmation, a per-workload DLP-event count, an activity sample, and any test-mode DLP drift flags — the raw material for the triage queue. Optionally exported to

CSV. .NOTES White-label. Read-only. Security & Compliance PowerShell / Exchange Online. No credentials in code. > [CmdletBinding()] param([int] $Days = 1, [string] $CsvPath)

OPSMANAGED-Remediation.ps1

Purpose: Remediates only the low-risk drift the service is allowed to fix directly — enabling a published-but-off label policy and enabling a disabled retention policy — WITHOUT ever switching a DLP policy to enforce or making any other high-impact change (those are routed to the owning capability and piloted in simulation/test there). Runs in -WhatIf by default; pass -Apply to make the two safe changes. It only REPORTS DLP policies still in test/audit for the owner to promote via the owning solution. It never deletes anything. HOW TO RUN Preview (safe, default): powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Remediation.ps1 Apply changes: powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Remediation.ps1 -Apply Sign in as a Compliance Administrator when prompted; make changes under change control (Req 2.6). EXAMPLE .\OPSMANAGED-Remediation.ps1 -WhatIf EXPECTED RESULTS A list of proposed actions (WhatIf) or the applied changes (-Apply). DLP enforce and all other high-impact changes are never automated — they are routed to the owning capability, piloted first. .NOTES White-label — Purview Solutions. Default is a dry run. Uses Get-/Set-LabelPolicy and Get-/Set-RetentionCompliancePolicy; reads Get-DlpCompliancePolicy. > [CmdletBinding()] param([switch]$Apply, [string]$LabelPolicyName, [string]$RetentionPolicyName) $WhatIf = -not $Apply $mode = $(if($WhatIf){'WHATIF (preview only)'}else{'APPLY'})

How to run: Preview (safe, default): powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Remediation.ps1 Apply changes: powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Remediation.ps1 -Apply Sign in as a Compliance Administrator when prompted; make changes under change control (Req 2.6). EXAMPLE .\OPSMANAGED-Remediation.ps1 -WhatIf EXPECTED RESULTS A list of proposed actions (WhatIf) or the applied changes (-Apply). DLP enforce and all other high-impact changes are never automated — they are routed to the owning capability, piloted first. .NOTES White-label — Purview Solutions. Default is a dry run. Uses Get-/Set-LabelPolicy and Get-/Set-RetentionCompliancePolicy; reads Get-DlpCompliancePolicy. > [CmdletBinding()] param([switch]$Apply, [string]$LabelPolicyName, [string]$RetentionPolicyName) $WhatIf = -not $Apply $mode = $(if($WhatIf){'WHATIF (preview only)'}else{'APPLY'})

OPSMANAGED-Report.ps1

Purpose: Produces `Managed Service – Service Report` for a cycle: it summarizes the triage CSV (from OPSMANAGED-Monitor.ps1 and the manual triage queue) into per-severity and per-workload counts, computes a simple SLO-adherence figure against a target response window, and exports the report as the baseline for the next cycle. Read-only — it summarizes and exports, it changes nothing. The portal is authoritative; reconcile before circulating. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Report.ps1 -TriageCsv .\opsmanaged-triage.csv powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Report.ps1 -TriageCsv .\in.csv -SloHours 8 -OutCsv .\service-report.csv Triage CSV columns (flexible): Severity, Workload, Policy, and optionally Opened/Closed timestamps and Owner for the SLO/ownership views. EXPECTED RESULTS Per-severity and per-workload counts, an SLO-adherence percentage (if Opened/Closed present), and (with -OutCsv) the report written out. PASS = the rollup reconciles with the portal and open items have an owner. .NOTES White-label. Read-only. No credentials in code. The portal is authoritative — reconcile before issuing. > [CmdletBinding()] param([Parameter(Mandatory=$true)][string] $TriageCsv, [double] $SloHours = 8, [string] $OutCsv)

How to run: powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Report.ps1 -TriageCsv .\opsmanaged-triage.csv powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Report.ps1 -TriageCsv .\in.csv -SloHours 8 -OutCsv .\service-report.csv Triage CSV columns (flexible): Severity, Workload, Policy, and optionally Opened/Closed timestamps and Owner for the SLO/ownership views. EXPECTED RESULTS Per-severity and per-workload counts, an SLO-adherence percentage (if Opened/Closed present), and (with -OutCsv) the report written out. PASS = the rollup reconciles with the portal and open items have an owner. .NOTES White-label. Read-only. No credentials in code. The portal is authoritative — reconcile before issuing. > [CmdletBinding()] param([Parameter(Mandatory=$true)][string] $TriageCsv, [double] $SloHours = 8, [string] $OutCsv)

OPSMANAGED-Validation.ps1

Purpose: Emits one PASS/FAIL line per operational health criterion so the service can validate the pilot and re-check each cycle: the unified audit log is on (the estate is evidenced), no intended DLP/ retention/label policy is missing/disabled/stuck in test, and the compliance role review is clean. Read-only — changes nothing. Use it at Implementation Step 5 and on the ongoing cadence. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Validation.ps1 Prereqs: ExchangeOnlineManagement (Connect-IPPSSession + Connect-ExchangeOnline), MFA-capable read admin. Verify capability availability against your licensing/tenant. EXPECTED RESULTS A list of PASS/FAIL lines and an overall pass count. Exit code 0 when all pass, 1 when any fail. Every FAIL is a triage item routed to its owning capability. .NOTES White-label. Read-only. Security & Compliance PowerShell / Exchange Online. No credentials in code. > [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\OPSMANAGED-Validation.ps1 Prereqs: ExchangeOnlineManagement (Connect-IPPSSession + Connect-ExchangeOnline), MFA-capable read admin. Verify capability availability against your licensing/tenant. EXPECTED RESULTS A list of PASS/FAIL lines and an overall pass count. Exit code 0 when all pass, 1 when any fail. Every FAIL is a triage item routed to its owning capability. .NOTES White-label. Read-only. Security & Compliance PowerShell / Exchange Online. No credentials in code. > [CmdletBinding()] param()